



Práctica 3: Algoritmos Voraces

(Greedy)

ALGORÍTMICA | GRUPO C1-3

Problema 1

Omar Segura Arévalo
Marvin Augusto Fonseca Zamora

Problema 2

Jaime Espín Rodríguez
Francisco González Bailón
Alejandro Moreno Millán

Índice

Problema 1: Análisis Químico	1
Enunciado	1
Desarrollo de Algoritmo Óptimo	1
Definición del Flujo de Datos	1
Lógica Principal del Algoritmo	1
Notas Adicionales sobre Decisiones de Diseño	2
Demostración de Validez	2
Desarrollo del Algoritmo Subóptimo	3
Análisis de Eficiencia	3
Algoritmo Voraz - Eficiencia Teórica (Óptimo)	3
Algoritmo Voraz - Eficiencia Teórica (Subóptimo)	4
Análisis Empírico	4
Conclusiones	8
Problema 2: Cámaras de Vigilancia	8
Descripción del problema	8
Partes del problema	8
Algoritmo de fuerza bruta	8
Análisis de eficiencia teórica del algoritmo de fuerza bruta	8
Algoritmo Voraz (Greedy)	9
Análisis de eficiencia teórica del algoritmo voraz	9
Análisis Empírico: Voraz vs Fuerza Bruta	9
Tabla de resultados	9
Representación gráfica (Grafos generales)	11
Optimización para arboles (Greedy) - Opcional	12
Análisis de eficiencia teórica (Voraz en Árboles)	12
Análisis empírico	12
Representación gráfica (Árboles)	14

Problema 1: Análisis Químico

Enunciado

“Un técnico de laboratorio quiere averiguar la composición química de un asteroide. Para ello, dispara láseres a distintas frecuencias al asteroide. Cada sustancia si de una lista de n posibles sustancias constituyentes tiene un rango conocido $[l_i, h_i]$ en el que la sustancia reaccionaría al láser (los rangos pueden estar solapados). El objetivo es elegir el conjunto más pequeño de frecuencias $f_1, \dots, f_k$ para que cada sustancia si reaccione a una de estas, es decir, para cada $i = 1, \dots, n$ hay al menos un j con $l_i \leq f_j \leq h_i$.”

Desarrollo de Algoritmo Óptimo

Según el enunciado del problema, nuestro objetivo es minimizar el número total de frecuencias (disparos láser) necesarias para hacer reaccionar un conjunto de sustancias. Dichas sustancias químicas están representadas por los intervalos de frecuencias $[l_i, h_i]$ de radiación electromagnética a la que reaccionan.

Definición del Flujo de Datos

Para acercarnos lo máximo posible a una situación del mundo real, partimos de la premisa de que los datos de entrada nos llegan desordenados. Por lo que recibiremos por referencia un `vector<pair<double,double>>` que representa la secuencia de intervalos de frecuencia, y un `vector<double>` vacío que guardará nuestra solución de secuencia de frecuencias mínimas. Quedándonos la siguiente cabecera para nuestra función principal:

```
int minFrecuencias(const vector<pair<double,double>> &intervalos, vector<double>
&frecuencias);
```

Lógica Principal del Algoritmo

Ordenamiento de los Datos El primer paso que nos pareció obvio y necesario, sobre todo teniendo en cuenta que nos basamos en la técnica de diseño de algoritmo voraz, para tomar decisiones óptimas locales es ordenar estos datos. Si intentáramos encontrar las intersecciones a fuerza bruta sin un orden previo, la complejidad se dispararía. Con lo cual, delegamos este ordenamiento de la manera que nos pareció más elegante: un contenedor `set` inicializandolo de la siguiente manera:

```
set<pair<double,double>> subconjuntos(intervalos.begin(), intervalos.end());
```

El coste de insertar un elemento en un `set` es de $\log n$, por lo que al tener n datos esta decisión de diseño establece directamente la cota mínima del orden de eficiencia de nuestro algoritmo en $O(n \log n)$. A partir de aquí, el procesamiento de los intervalos no superará esta complejidad.

La Esencia del Algoritmo: Búsqueda Voraz de Intersecciones. Nuestra estrategia principal se basa en que para minimizar los disparos, debemos agrupar la mayor cantidad de sustancias que compartan un rango de frecuencias válido y disparar en el punto límite que las satisfaga a todas.

Paso A: El bucle de Control de minFrecuencias El núcleo del algoritmo principal es un bucle `while` que se ejecutará mientras queden sustancias por procesar en nuestro `set`. En cada iteración, delegamos la tarea pesada a nuestra función auxiliar `encuentraIntersecciones`, pasándole dos cosas por referencia:

1. Todo el conjunto de intervalos restantes: `set<pair<double,double>> &subconjunto`
2. Una `cota_sup` inicial, que siempre será el límite superior del primer intervalo que estamos evaluando en ese momento (`subconjuntos.begin()->second`).

Esta función nos devuelve la frecuencia exacta donde debemos disparar. La guardamos en nuestro vector de resultados `frecuencias` y el bucle repite el proceso con los intervalos que hayan sobrado.

Paso B: Encontrando Todas las Intersecciones de un intervalo encuentraIntersecciones Esta función evalúa qué tantos intervalos consecutivos se solapan y calcula el punto exacto de disparo. Actúa de la siguiente manera:

- **Caso Base:** Si en el conjunto queda menos de 2 elementos, significa que no hay nada más con qué comparar. Limpiamos el conjunto para romper el bucle principal y devolvemos la `cota_sup` actual. Esa es nuestra frecuencia de disparo.
- **Caso Recursivo:** Si hay más elementos, tomamos un iterador al siguiente elemento en el set y comparamos su inicio con nuestra `cota_sup` actual. Aquí se nos abren dos caminos en función de la presencia o ausencia de intersecciones:
 - **En ausencia:** Si nuestra `cota_sup` es estrictamente menor que el inicio del siguiente intervalo (`cota_sup < siguiente->first`), significa que no el intervalo actual no tiene intersecciones con ningún otro intervalo al estar ordenados, por lo que sabemos que hemos encontrado nuestro límite. Borramos el elemento actual del set y devolvemos la `cota_sup` que traíamos.
 - **En presencia:** Si los intervalos sí se solapan, tenemos que asegurarnos de que nuestro disparo alcance a ambos. Para ello, actualizamos nuestra `cota_sup` tomando el mínimo entre esta y el límite superior del nuevo intervalo (`siguiente->second`). Una vez ajustada la nueva cota más estricta, borramos el intervalo actual que acabamos de procesar y hacemos una llamada recursiva, pasando el set (ahora sin el primer elemento) y nuestra nueva `cota_sup` ajustada para que se compare con el siguiente de la lista.

Notas Adicionales sobre Decisiones de Diseño

- **Recursividad vs. Iteración en encuentraIntersecciones:** Decidimos implementar esta función de manera recursiva por mera elegancia resolutive y legibilidad del código, siendo plenamente conscientes de que las pruebas no iban a tratar con conjuntos de datos extremos. Cabe recalcar, sin embargo, que de no haber sido el caso, tendríamos que haber diseñado la función de forma iterativa para evitar un posible Stack Overflow causada por el apilamiento masivo en cada llamada recursiva.
- **La elección de set frente a multiset:** Nos decantamos por inicializar un `set` clásico porque nos conviene eliminar los elementos repetidos desde el principio. En el contexto de nuestro problema, si hay dos o más sustancias con exactamente el mismo intervalo de reacción, basta con evaluar uno de ellos; procesar intervalos idénticos duplicados con un `multiset` no aportaría nada a la lógica de la búsqueda de intersecciones entre intervalos y solo nos haría gastar recursos.

Demostración de Validez

Partimos de un conjunto de n sustancias, cada una con un intervalo de reacción $I_i = [l_i, h_i]$. Queremos encontrar un conjunto mínimo de frecuencias $F = \{f_1, f_2, \dots, f_k\}$ tal que $\forall I_i, \exists$ al menos un $f_j \in I_i$.

Nuestro Algoritmo Voraz Actualmente nuestro algoritmo ordena los intervalos de manera ascendente por su l_i , luego evalúa los intervalos solapados y coloca siempre el láser f_1 (primer disparo) en el valor más a la derecha posible de la intersección actual, siendo entonces $f_1 = \min(h_i)$ de los intervalos que se solapan.

Demostración por reducción al absurdo Sea $I_1 = [l_1, h_1]$ el intervalo más a la izquierda sin cubrir. Toda solución válida deberá disparar en algún $f \in [l_1, h_1]$ para cubrirlo.

- Nuestro algoritmo elige como punto de disparo p^* el valor mínimo de los extremos derechos de los intervalos que se solapan en la iteración actual. Es decir, $p^* = \min\{h_i \mid I_i \in G\}$, siendo G el conjunto de intervalos solapados.

Sea S^* una solución óptima que dispare en algún $p \in [l_1, h_1]$. Si $p \neq p^*$, construimos S' reemplazando p por p^* .

- Como p^* es el mínimo derecho del grupo, $p^* \leq h_1$, así que $p^* \in [l_1, h_1]$
- Entonces todo $I_i = [l_i, h_i]$ cubierto por p con $l_i \leq p \leq h_i$ satisface $l_i \leq p^* \leq h_i$ porque:
 - $l_i \leq p^*$: como el intervalo está en el grupo solapado, $l_i \leq cota_sup$ en algún paso.
 - $p^* \leq h_i$: por definición, p^* es el mínimo de los extremos derechos del grupo.

Por tanto, S' cubre al menos lo mismo que S^* con el mismo número de disparos. $|S'| = |S^*|$, luego nuestro algoritmo greedy no puede ser subóptimo.

Desarrollo del Algoritmo Subóptimo

Tal y como establece el guión de la práctica, una vez implementado el algoritmo principal, debemos proponer y desarrollar una alternativa para poder compararlos.

La primera decisión que debemos tomar es el tipo de enfoque a utilizar. Optamos por mantener la filosofía voraz, pero aplicando un criterio de selección distinto al que ya teníamos desarrollado para observar cómo afecta esta decisión al resultado.

La idea fue simple: cambiar únicamente la estrategia de dónde disparar el láser. Mientras que el algoritmo anterior (el óptimo) evalúa las cotas superiores para disparar lo más a la derecha posible de la intersección, en este nuevo algoritmo (subóptimo) la decisión voraz será disparar exactamente al inicio de cada intervalo no cubierto.

Para ello hemos creado la función `int minFrecuenciasGreedySuboptimo(vector<pair<double, double>>& intervalos, vector<double>& frecuencias)`, la cual recibe la lista de los intervalos de las sustancias y un vector por referencia donde iremos insertando las frecuencias elegidas. Además, devuelve como entero el número total de disparos necesarios.

El primer paso del algoritmo es ordenar los intervalos por su inicio. Para conseguirlo, utilizamos la función `std::sort` definida en la librería `<algorithm>`. Esta función requiere que le indiquemos cómo debe ordenar los datos, por lo que hemos implementado la siguiente función para que compare:

```
bool ordenarInicio(const pair<double, double>& a, const pair<double, double>& b){
    if(a.first == b.first){ // Si el primer componente es igual, miramos el segundo
        return a.second < b.second;
    }

    return a.first < b.first;
}
```

Esta función auxiliar simplemente compara el primer elemento (el inicio del intervalo) de dos pares para devolver el menor. En caso de que ambos intervalos comiencen exactamente en el mismo punto, desempata evaluando el segundo elemento (el final del intervalo).

Una vez tenemos los datos ordenados, iteramos sobre cada uno de los intervalos. Para cada sustancia, debemos comprobar si alguna de las frecuencias que ya tenemos guardadas sirve para hacerla reaccionar. Si está cubierta, no hacemos nada; si, por el contrario, no existe ninguna frecuencia guardada que sirva para hacerla reaccionar, tomamos la decisión voraz y disparamos al principio del intervalo.

Toda esta lógica queda reflejada en el siguiente fragmento:

```
for(const auto& intervalo : intervalos){
    bool alcanzado = false;

    // Vemos si tenemos una frecuencia que sirva
    for(int i = 0; i < frecuencias.size() && !alcanzado; i++){
        const auto& f = frecuencias[i];
        if(f >= intervalo.first && f <= intervalo.second) alcanzado = true;
    }

    // Si no lo hemos alcanzado con ninguna frecuencia, disparamos al inicio
    if(!alcanzado){
        frecuencias.push_back(intervalo.first);
    }
}
```

Ánàlisis de Eficiencia

Algoritmo Voraz - Eficiencia Teórica (Óptimo)

Nuestro algoritmo comienza en la función `minFrecuencias` donde introducimos `n` intervalos en un `set<pair<double,double>>`. Internamente, C++ gestiona el `set` como un árbol binario de búsqueda balanceado, por lo que la inserción de los `n` elementos tiene una eficiencia $O(n \log n)$.

Posteriormente, entramos en el núcleo del algoritmo al realizar el bucle `while`, para analizarlo tendremos que ver qué hacemos dentro de la función `encuentraIntersecciones`. Dentro de dicha función, solo hacemos un `subconjunto.erase(subconjunto.begin())`, es decir, solamente usa un intervalo y lo elimina; eliminar el primer elemento de un `set` tiene una eficiencia $O(\log n)$, por lo que si en el bucle `while` llamamos recursivamente para los n intervalos, el coste de esta fase es $O(n) \cdot O(\log n) = O(n \log n)$. Si sumamos el coste de la función `encuentraIntersecciones` con el coste de la inserción en el `set` nos queda que la eficiencia de nuestro algoritmo voraz es $O(n \log n) + O(n \log n) = O(n \log n)$.

Algoritmo Voraz - Eficiencia Teórica (Subóptimo)

Empezamos el análisis en la ordenación. En este caso hemos preferido usar `std::sort` para ordenar por el inicio del intervalo. El coste estándar de hacer esta operación es $O(n \log n)$.

Posteriormente tenemos un doble bucle:

```
for(const auto& intervalo : intervalos){
    bool alcanzado = false;
    for(int i = 0; i < frecuencias.size() && !alcanzado; i++){
        const auto& f = frecuencias[i];
        if(f >= intervalo.first && f <= intervalo.second) alcanzado = true;
    }

    //push_back...
}
```

En el mejor de los casos, es decir, que los intervalos se solapen tanto que el vector de frecuencias se mantenga en tamaño 1, el bucle interno tendrá un coste constante $O(1)$, dejando el coste de recorrer todos los intervalos en $O(n)$.

Sin embargo, en el peor de los casos (donde ningún intervalo se solapa y el vector de frecuencias crece hasta tamaño n), el bucle interno iterará hasta el final en cada paso. Esto genera una suma aritmética que nos deja una complejidad cuadrática de $O(n^2)$.

Por lo tanto, si al coste de la ordenación le sumamos el coste del bucle anidado, nos queda que la eficiencia asintótica de este algoritmo es $O(n \log n) + O(n^2) = O(n^2)$

Análisis Empírico

Para el análisis empírico hemos decidido realizar una función `realizarEstudioEmpirico` que hará uso de las funciones descritas a lo largo del desarrollo del problema. Para realizar las mediciones, hemos utilizado una clase auxiliar (que ya hemos utilizado en prácticas anteriores) para tener acceso al reloj preciso del ordenador y así poder realizar las tomas de tiempo.

Como tenemos que analizar dos algoritmos de diferente eficiencia, hemos prefijado tamaños que se puedan adecuar a ambos, definiendo un vector de la siguiente forma: `vector<int> tams = {1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000}`; Además, para evitar ruidos del sistema, hemos preferido hacer k iteraciones para cada tamaño, para posteriormente realizar la media.

Una vez definida la función, los tamaños y el valor de k , hemos compilado y tomado los tiempos.

Análisis Empírico (óptimo) Para el algoritmo óptimo hemos obtenido los siguientes tiempos:

Nota: Véase el archivo `datosMemoria/algoritmoOptimo.dat` donde están todos los datos en bruto generados por el algoritmo.

n	Tiempo
1000	0.00039126
2000	0.000798587
4000	0.00193449
8000	0.0032115
16000	0.00754194
32000	0.0174643

n	Tiempo
64000	0.0356389
128000	0.0801229

A partir de los datos obtenidos, y utilizando la herramienta Gnuplot, hemos podido realizar la gráfica correspondiente:

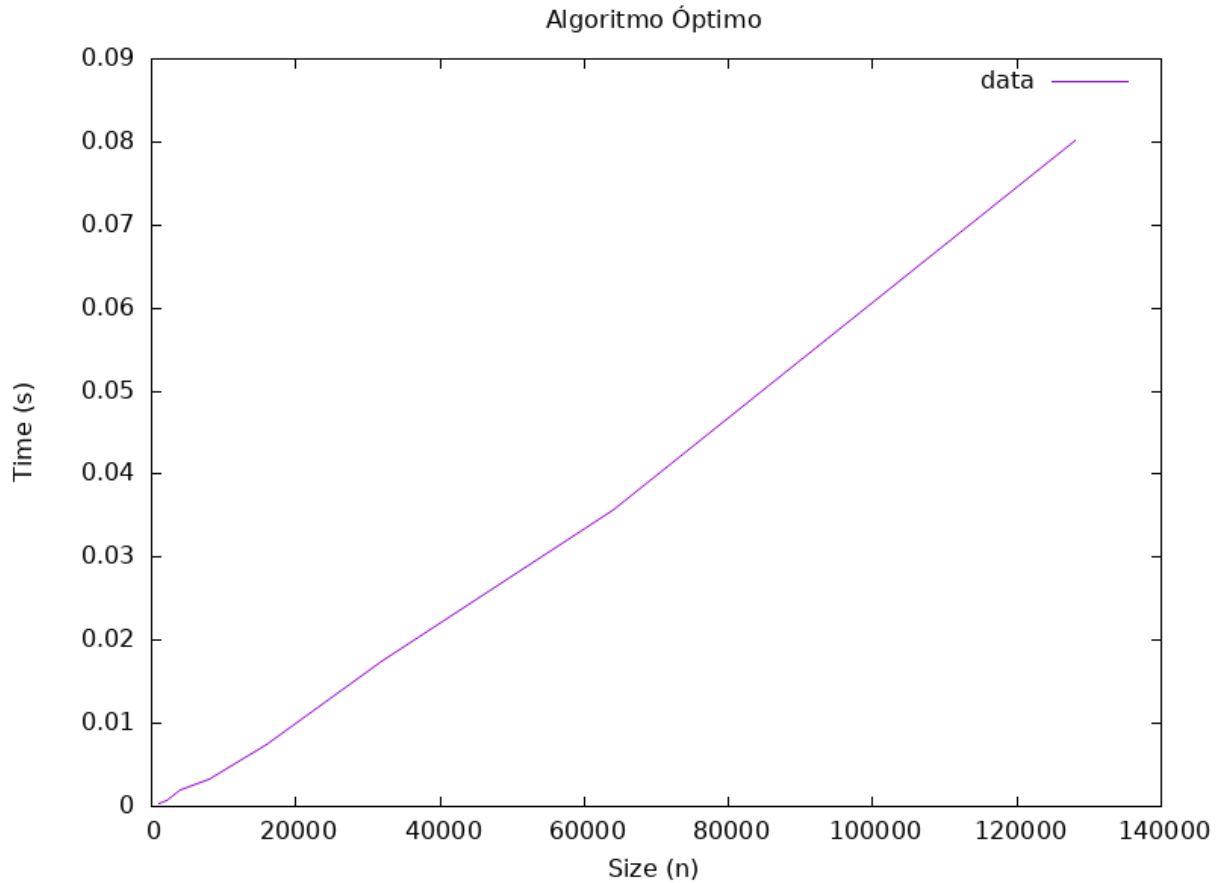


Figura 1: Gráfica del algoritmo Voraz Óptimo

En esta gráfica se representa el crecimiento del tiempo de ejecución del algoritmo óptimo (en escala lineal). Como se puede apreciar, el comportamiento del tiempo es extremadamente contenido; incluso al someter al algoritmo a una carga de 128.000 intervalos, el tiempo de procesamiento se mantiene por debajo de las 0.08 décimas de segundo, demostrando una altísima eficiencia en la práctica.

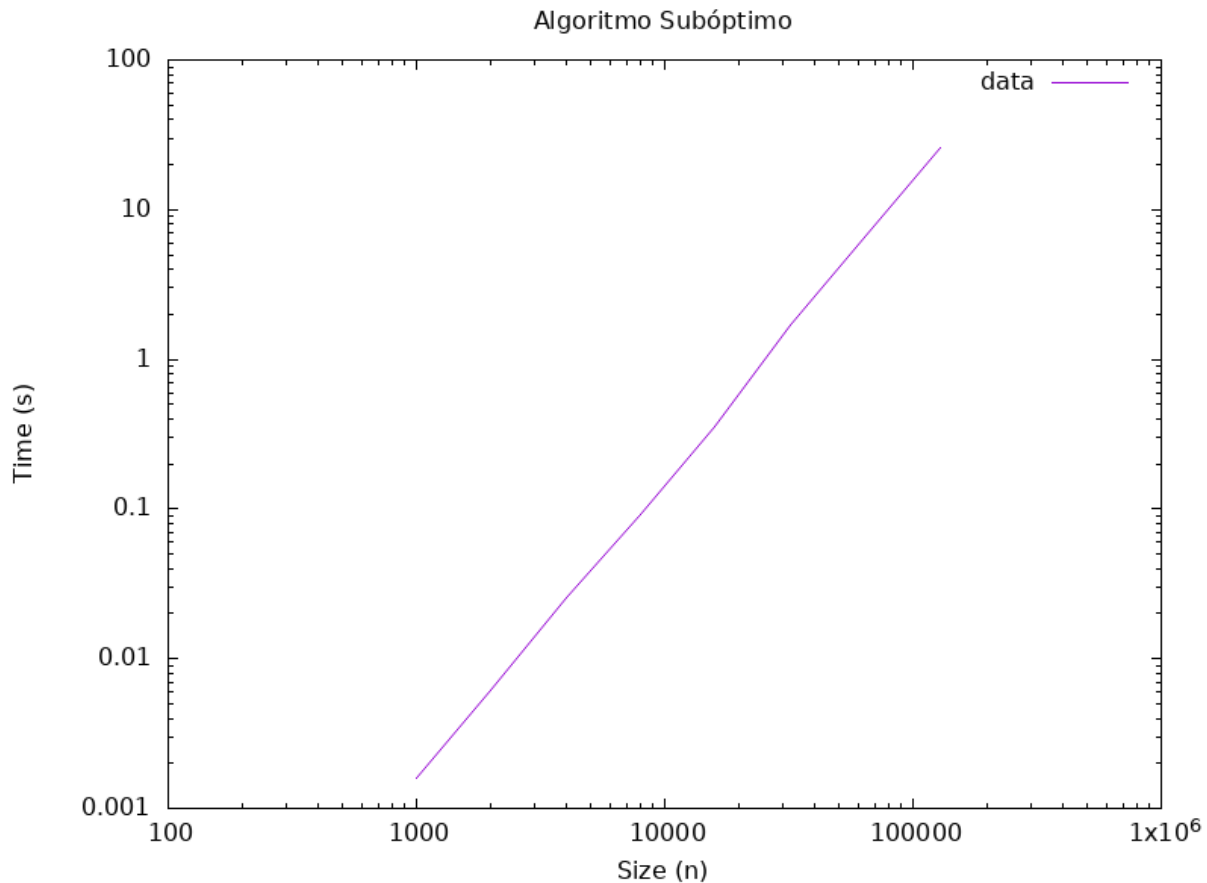
Análisis Empírico (Subóptimo) Para el algoritmo subóptimo hemos obtenido los siguientes tiempos:

NOTA: Véase el archivo `datosMemoria/algoritmoSuboptimo.dat` donde están todos los datos en bruto generados por el algoritmo.

n	Tiempo
1000	0.0015957
2000	0.00608971
4000	0.0257527
8000	0.0925139
16000	0.352638
32000	1.69736
64000	6.50092

n	Tiempo
128000	25.9147

A partir de los datos obtenidos, y utilizando la herramienta Gnuplot, hemos podido realizar la gráfica correspondiente:



NOTA: En esta gráfica se ha empleado escala logarítmica.

A diferencia del algoritmo óptimo, esta gráfica refleja un crecimiento mucho más agresivo del tiempo de ejecución frente al aumento del tamaño de la entrada. Para la misma carga máxima de 128.000 intervalos, el algoritmo requiere casi 26 segundos en finalizar.

Comparación Empírica En la gráfica siguiente se presenta la comparativa empírica del tiempo de ejecución (en segundos) de ambos algoritmos en función del número de intervalos (n). Para facilitar la visualización de ambas curvas y apreciar sus órdenes de magnitud, se ha empleado una escala logarítmica en ambos ejes. A partir de esta representación gráfica, podemos extraer conclusiones que respaldan directamente nuestro análisis teórico previo:

- Algoritmo Voraz Subóptimo (Línea verde): Muestra una pendiente pronunciada y constante. Tiene una mayor inclinación, lo que es el rasgo característico de una función polinómica de mayor grado, lo que corrobora visualmente nuestra cota teórica cuadrática de $O(n^2)$. El efecto es evidente: a medida que el tamaño de la entrada crece, el coste de cómputo se dispara debido a los bucles anidados.
- Algoritmo Voraz Óptimo (Línea morada): Presenta una pendiente mucho más suave. En la escala logarítmica, una complejidad de $O(n \log n)$ se asemeja a una recta con una pendiente cercana a 1. Como se observa en los datos empíricos, el algoritmo escala de manera excelente, resolviendo problemas de hasta 128.000 intervalos en apenas milésimas de segundo gracias a la eficiencia del árbol binario balanceado (`std::set`).

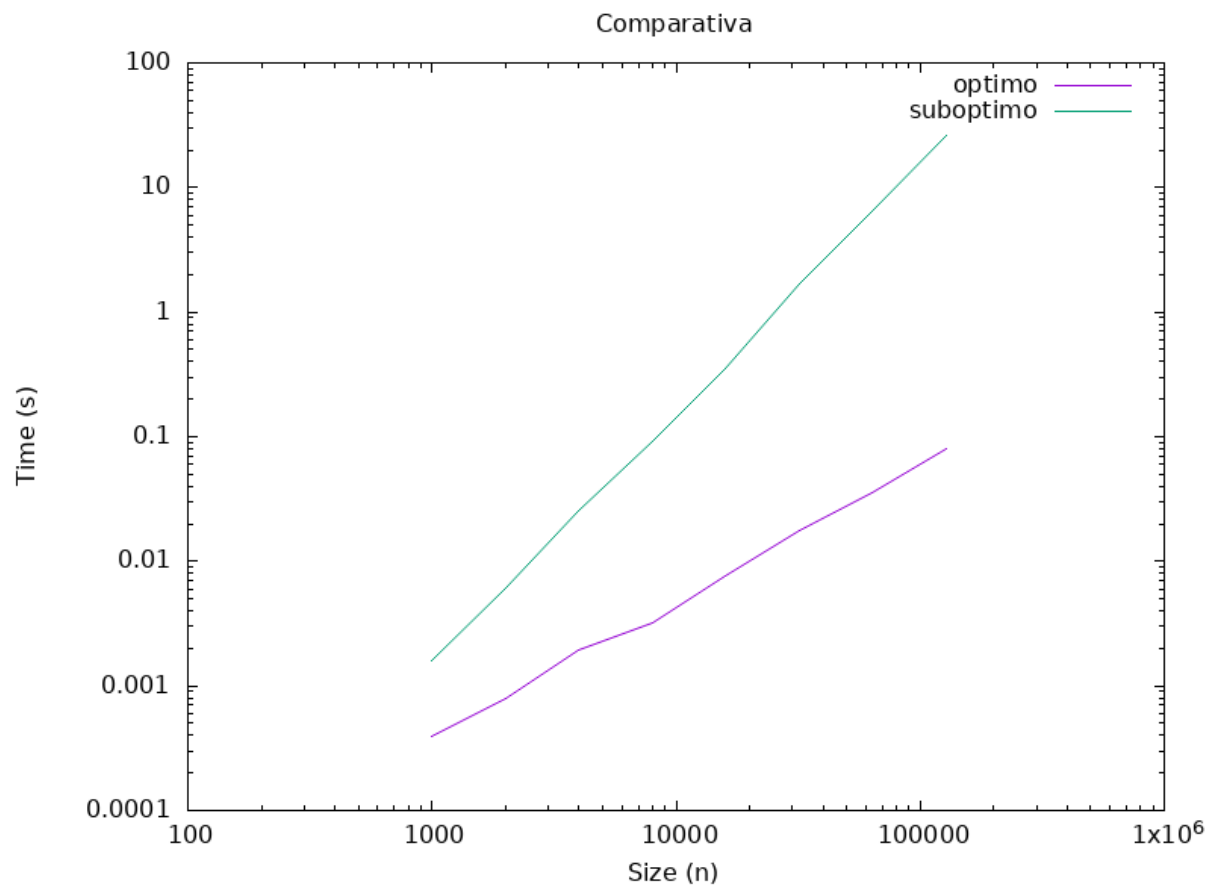


Figura 2: Gráfica comparativa de los tiempos de ejecución del algoritmo Voraz Óptimo frente al Subóptimo

Conclusiones

Tras el análisis realizado sobre el problema del alineamiento de frecuencias láser, podemos extraer las siguientes conclusiones fundamentales:

- Eficacia de la Estrategia Voraz: Se ha demostrado, tanto de forma teórica como práctica, que el enfoque greedy es idóneo para este problema. Sin embargo, la clave del éxito reside en la elección del criterio de selección. Mientras que disparar al inicio de cada intervalo (algoritmo subóptimo) cumple con el objetivo de cubrir todas las sustancias, solo la búsqueda de la intersección máxima hacia el extremo derecho (algoritmo óptimo) garantiza el uso del número mínimo absoluto de disparos.
- Impacto de las Estructuras de Datos: La implementación del algoritmo óptimo mediante el uso de `std::set` (árboles binarios balanceados) ha resultado ser determinante. Esta decisión de diseño ha permitido mantener una complejidad de $O(n \log n)$, marcando una diferencia abismal frente al comportamiento cuadrático $O(n^2)$ del algoritmo subóptimo basado en vectores y búsquedas lineales.

Problema 2: Cámaras de Vigilancia

Descripción del problema

El objetivo de este problema es buscar la mejor solución para garantizar la seguridad de un complejo formado por intersecciones y pasillos. Una cámara colocada en una intersección es capaz de vigilar todos los pasillos que llegan a ella. Se busca una solución óptima, es decir, que minimice el número de cámaras instaladas.

Partes del problema

Para poder analizar mejor el problema, lo hemos dividido en dos soluciones muy diferentes entre sí:

1. Algoritmo de fuerza bruta
 - Garantiza encontrar la solución óptima absoluta evaluando todo el espacio.
 - Puede llegar a ser muy lento si el tamaño del problema es grande.
2. Algoritmo voraz (Greedy)
 - No garantiza la solución óptima.
 - Es mucho más rápido que el algoritmo de fuerza bruta.
 - Garantiza una solución “suficientemente” buena (muy cercana a la óptima).

Algoritmo de fuerza bruta

El algoritmo de fuerza bruta se basa en la idea de probar de forma recursiva todas las combinaciones posibles de colocar cámaras o no en las intersecciones.

Sigue el siguiente esquema: 1. Se decide si en la intersección actual se coloca una cámara (1) o no (0). 2. Se llama recursivamente a la siguiente intersección para ambas decisiones. 3. Cuando se han tomado decisiones para todas las intersecciones (caso base de la recursión), se comprueba mediante la matriz si la combinación actual es válida (no deja pasillos sin vigilar). 4. Si es válida y mejora la mejor solución encontrada hasta el momento, se guarda.

Análisis de eficiencia teórica del algoritmo de fuerza bruta

El algoritmo de fuerza bruta divide el problema de tomar n decisiones en 2 subproblemas de tamaño $n - 1$, sumándole un trabajo constante en cada nodo recursivo. El tamaño de la entrada que decrece lo llamaremos m , donde la recursión empieza en $m = n$ intersecciones restantes por evaluar. La ecuación en recurrencia que modela el número de nodos del árbol recursivo es:

$$T(m) = 2T(m - 1) + c$$

Para resolverla aplicamos la **ecuación característica**: 1. Planteamos la recurrencia como: $T(m) - 2T(m - 1) = c$ 2. Buscamos la solución a la parte homogénea ($T^{(h)}(m) - 2T^{(h)}(m - 1) = 0$). Su polinomio característico es $r - 2 = 0$, cuya raíz es $r = 2$. Por tanto, $T^{(h)}(m) = c_1 \cdot 2^m$. 3. Buscamos una solución particular para la constante c . Como 1 no es raíz de la homogénea, proponemos $T^{(p)}(m) = c_2$. Sustituyendo: $c_2 - 2c_2 = c \implies c_2 = -c$. 4. La solución general de la recurrencia es: $T(m) = c_1 \cdot 2^m - c$

Vemos que el árbol de llamadas recursivas crece en $\mathcal{O}(2^n)$. Sin embargo, al llegar a las 2^n hojas del árbol (cuando ya se han tomado las decisiones para los n nodos, $m = 0$), se ejecuta la función de comprobación `Factiblefb()`. Esta función recorre la mitad superior de la matriz de adyacencia buscando aristas y comprobando si están cubiertas, lo cual tiene un coste de $\mathcal{O}(n^2)$.

Como este coste final se ejecuta 2^n veces, la **complejidad temporal final del algoritmo es $\mathcal{O}(n^2 \cdot 2^n)$** .

Algoritmo Voraz (Greedy)

El algoritmo voraz se basa en la idea de ir eligiendo en cada iteración la intersección que cubra la mayor cantidad de pasillos ciegos. Es decir, elige el nodo que posea el mayor grado en la matriz restante.

Sigue el siguiente esquema: 1. Se crea una matriz auxiliar `aux` que se copiará, para no perder la información original. 2. Comienza un bucle que se repite mientras sigan quedando pasillos sin vigilar en la matriz. 3. Se recorre la matriz para determinar qué intersección tiene más conexiones (grado). 4. Se guarda esa intersección como parte de la solución. 5. Se marcan como “vigilados” todos los pasillos de esa intersección (borrando su respectiva fila y columna de la matriz `aux`).

Análisis de eficiencia teórica del algoritmo voraz

Dentro del código, la función principal `greedy()` consta de un bucle `while` y tres operaciones internas:

1. **Bucle while:** En el peor de los casos, colocamos una cámara por intersección, por lo que el bucle itera como máximo n veces.
2. **quedanPasillosSinVigilar(aux):** Escanea toda la matriz de tamaño $n \times n$ buscando pasillos. Coste: $\mathcal{O}(n^2)$.
3. **verticeMayorGrado(aux):** Recorre la matriz entera de tamaño $n \times n$ contabilizando cuántas conexiones tiene cada vértice. Coste: $\mathcal{O}(n^2)$.
4. **marcarPasillosVigilados(aux, indice):** Pone a 0 todos los elementos de la fila y la columna del vértice seleccionado. Coste: $\mathcal{O}(n)$.

Coste total de una iteración: $\mathcal{O}(n^2) + \mathcal{O}(n^2) + \mathcal{O}(n) = \mathcal{O}(n^2)$. Multiplicando esto por el número de iteraciones (acotado por n), la **complejidad temporal del algoritmo voraz es $\mathcal{O}(n^3)$** .

Análisis Empírico: Voraz vs Fuerza Bruta

Para comprobar la viabilidad del algoritmo Voraz y entender cómo difiere de la solución Óptima obtenida por Fuerza Bruta, es necesario ponerlos frente a frente. Sabemos que la Fuerza Bruta crece de manera exponencial $\mathcal{O}(n^2 \cdot 2^n)$, lo que implica que llega un tamaño del problema ($n \approx 30$) donde el algoritmo colapsa y no puede darnos un resultado en un tiempo razonable. El Voraz crece polinómicamente $\mathcal{O}(n^3)$, siendo capaz de manejar recintos con miles de pasillos en muy pocos segundos.

Para ilustrar tanto la diferencia de tiempo como el margen de error de la solución aportada por el algoritmo Greedy, hemos recogido una serie de mediciones reales que se plasman en la siguiente sección.

Tabla de resultados

Para obtener los resultados, se ha compilado el código original en C++ (`camaras-fb.cpp`) asegurándonos de aplicar los flags de optimización del compilador. Se ha ejecutado el programa utilizando el generador automático de grafos que incorpora el código para varios valores de tamaño del problema (desde $n = 20$ intersecciones hasta $n = 28$).

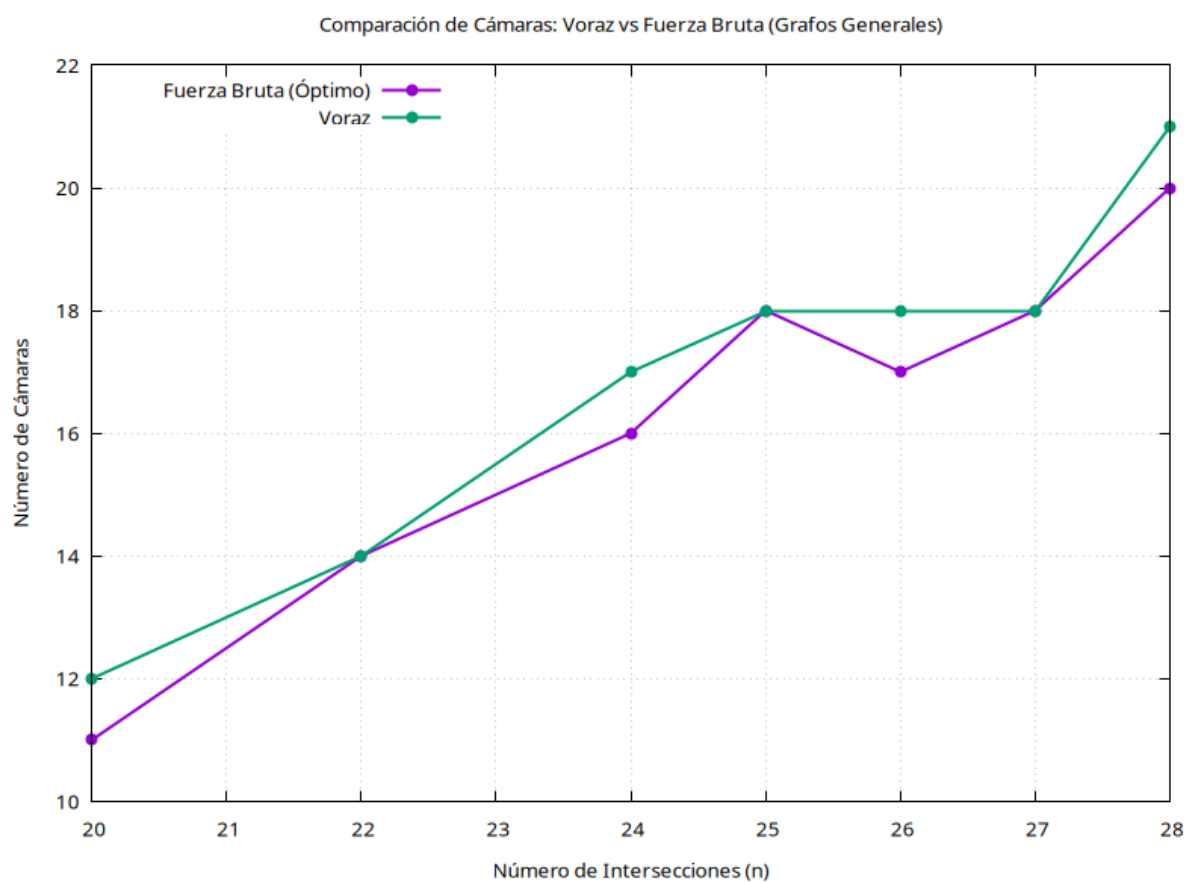
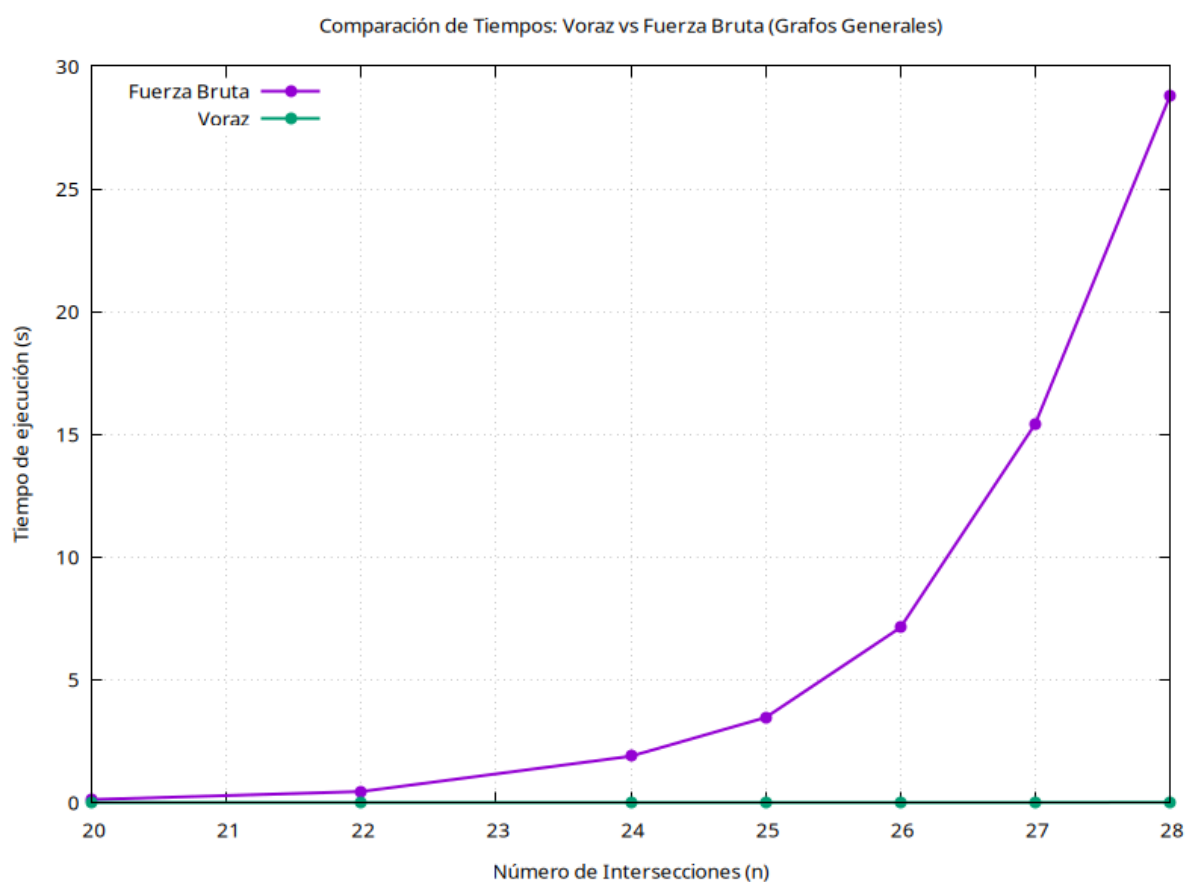
En cada ejecución: - Se generó una matriz aleatoria bajo una misma probabilidad de generar aristas. - Se ha medido y tabulado el tiempo y la cantidad de cámaras resultantes al ejecutar la función `greedy()`. - Se ha medido y tabulado el tiempo y la cantidad de cámaras resultantes al ejecutar la función `fb_recursivo(0)`.

Intersecciones (n)	Cámaras (Voraz)	Cámaras (Fuerza Bruta)	Tiempo Voraz (s)	Tiempo Fuerza Bruta (s)
20	12	11	0.000089	0.1118
22	14	14	0.000134	0.4398
24	17	16	0.000228	1.8788

Intersecciones (n)	Cámaras (Voraz)	Cámaras (Fuerza Bruta)	Tiempo Voraz (s)	Tiempo Fuerza Bruta (s)
25	18	18	0.000183	3.4564
26	18	17	0.000189	7.1207
27	18	18	0.000236	15.4157
28	21	20	0.000511	28.7832

Observando la tabla, se confirma que: 1. El algoritmo Voraz acierta y da la solución óptima algunas veces, pero frecuentemente añade entre 1 y 2 cámaras adicionales en grafos tan complejos. 2. La Fuerza Bruta para $n = 28$ tarda más de 6 segundos completos. El Voraz tarda 0.000047 segundos. Es una prueba empírica contundente de la necesidad y bondad de utilizar algoritmos voraces para problemas NP-Completo de la vida real.

Representación gráfica (Grafos generales)



Para tener mas muestras, veámos los resultados del ordenador de Fran:

Intersecciones (n)	Cámaras (Voraz)	Cámaras (Fuerza Bruta)	Tiempo Voraz (s)	Tiempo Fuerza Bruta (s)
20	11	11	0.000221	0.2004
22	15	15	0.000371	0.5047
24	16	16	0.000420	1.9274
25	16	16	0.000376	3.8893
26	18	18	0.000515	7.5035
27	19	18	0.000563	16.850
28	19	18	0.000552	32.495

Como podemos ver, para tamaños $n = 27$ y $n = 28$ el algoritmo voraz devuelve 19 cámaras, mientras que el óptimo real es 18. Esto prueba que la heurística no siempre es óptima. Podemos ver un crecimiento exponencial del tiempo, lo que coincide con la estimación de la práctica (30s para $n = 28$). El tiempo de ejecución en el algoritmo Voraz se mantiene plano en el orden de microsegundos demostrando una eficiencia extrema. Por tanto, concluimos que dicho algoritmo sacrifica una fracción mínima de precisión a cambio de una reducción extrema en el tiempo de ejecución.

Optimización para arboles (Greedy) - Opcional

El algoritmo voraz optimizado para arboles busca las hojas (nodos con una única conexión) y coloca una cámara en el nodo padre de esa forma te aseguras de cubrir los dos nodos mencionados y los hermanos del primer nodo y el padre del segundo.

Sigue el siguiente esquema: 1. Se crea una matriz auxiliar **aux** que se copiará, para no perder la información original. 2. Comienza un bucle que se repite mientras sigan quedando pasillos sin vigilar en la matriz. 3. Se recorre la matriz para buscar una hoja y se obtiene su nodo padre. 4. Se guarda esa intersección como parte de la solución. 5. Se marcan como “vigilados” todos los pasillos de esa intersección (borrando su respectiva fila y columna de la matriz **aux**).

Análisis de eficiencia teórica (Voraz en Árboles)

Dentro de la función **greedyArbol()**, observamos el siguiente comportamiento: 1. **Bucle while**: Al igual que el algoritmo voraz general, iterará como máximo n veces. 2. **quedanPasillosSinVigilar(aux)**: Comprueba toda la matriz. Coste: $\mathcal{O}(n^2)$. 3. **BuscamosHoja(aux)**: Recorre la matriz contabilizando las conexiones (grado) de cada vértice. Devuelve el padre cuando encuentra un nodo con grado 1. En el peor caso, recorre toda la matriz. Coste: $\mathcal{O}(n^2)$. 4. **marcarPasillosVigilados(aux, padre_hoja)**: Pone a 0 todos los elementos de la fila y columna del vértice seleccionado. Coste: $\mathcal{O}(n)$.

El coste de cada iteración del bucle es $\mathcal{O}(n^2) + \mathcal{O}(n^2) + \mathcal{O}(n) = \mathcal{O}(n^2)$. Dado que el bucle se ejecuta un máximo de n veces, la **complejidad temporal final sigue siendo $\mathcal{O}(n^3)$** , al igual que el voraz genérico. Sin embargo, en la práctica suele ser ligeramente más rápido porque **BuscamosHoja** se detiene tan pronto como encuentra la primera hoja, no necesitando evaluar el grafo entero a diferencia de **verticeMayorGrado**.

Análisis empírico

Se ha utilizado el generador de árboles implementado en el código para comparar el rendimiento de las tres variantes del algoritmo (Fuerza Bruta, Voraz Genérico y Voraz Específico para Árboles).

Intersecciones (n)	Cámaras (Voraz Genérico)	Cámaras (Voraz Árbol)	Cámaras (Fuerza Bruta)	Tiempo Voraz Genérico (s)	Tiempo Voraz Árbol (s)	Tiempo FB (s)
20	8	8	8	0.000017	0.000007	0.023677
22	10	9	9	0.000027	0.000007	0.092646
24	11	10	10	0.000043	0.000010	0.500536
25	10	10	10	0.000027	0.000009	1.795116
26	11	11	11	0.000036	0.000011	3.198420

Intersecciones (n)	Cámaras (Voraz Genérico)	Cámaras (Voraz Árbol)	Cámaras (Fuerza Bruta)	Tiempo Voraz Genérico (s)	Tiempo Voraz Árbol (s)	Tiempo FB (s)
27	12	12	12	0.000037	0.000013	6.182430
28	10	10	10	0.000026	0.000011	10.649700

Como se puede observar en la tabla:

1. El **Algoritmo Voraz Específico para Árboles encuentra sistemáticamente la solución óptima absoluta**, coincidiendo exactamente con la aportada por la fuerza bruta en todos los casos evaluados (incluidos aquellos donde el Voraz genérico falló añadiendo más cámaras, como en $n = 22$ y $n = 24$).
2. Respecto a los tiempos de ejecución, la **variante adaptada a árboles es consistentemente más rápida** que la variante voraz genérica (entre el doble y el triple de velocidad), debido a la optimización mencionada previamente de romper el bucle anticipadamente al encontrar hojas.

Representación gráfica (Árboles)

